Clase 7 · Tres formas de controlar un periférico en la Pico

Organización y Arquitectura de Computadoras · M.I. José Antonio de Jesús Arredondo Garza · lunes 28 sep 2026 · Tema 3 · Apuntes y resumen de estudio

PARTE 1 · Apuntes completos

Librerías de MicroPython vs. registros con `mem32` vs. ensamblador inline ARM Thumb, más campos de bits con `uctypes`.

1. Blink con librerías y el costo oculto en ciclos

```
from machine import Pin
from time import sleep
pin25 = Pin(25, Pin.OUT)        # LED integrado
while True:
    pin25.toggle()
    sleep(0.125)
```

> **El problema de las librerías**
>
> Son **opacas**: no se sabe cuántos ciclos consumen. A bajo nivel, la configuración del puerto se **reejecuta en cada vuelta** del `while True`, agregando **al menos 4 ciclos** por iteración. No sirven para temporización determinista.

2. Registros directos con machine.mem32

Dirección absoluta = **base + offset**.

Elemento	Pico 1	Pico 2 (RP2350)
Base de IO_BANK0	**0x40014000**	**0x40000000** (según los apuntes; verificar en el datasheet)
Base del SIO	0xd0000000	0xd0000000

- **GPIO25_CTRL** = base + **0x060**; se escribe la **función 5**, que conecta el pin al **SIO** (Single Input Output: acceso rápido de un ciclo).
- **GPIO_OUT_ENABLE_SET** (SIO + **0x024**): configurar como salida con máscara `1 << 25`.
- **GPIO_OUT_XOR** (SIO + **0x01c**): invertir el estado escribiendo `1 << 25`, sin leer antes el registro (operación atómica).

```
from machine import mem32
import time
BASE_PADS = 0x40014000           # Pico 2: 0x40000000
SIO_BASE  = 0xd0000000
mem32[BASE_PADS + 0x060] = 5                 # GPIO25 → SIO
mem32[SIO_BASE + 0x024] = (1 << 25)          # salida
while True:
    mem32[SIO_BASE + 0x01c] = (1 << 25)      # toggle
    time.sleep_ms(125)
```

La configuración toma **5 ciclos fijos**, una sola vez al inicio (1 dirección base, 2–3 direcciones secundarias, 1 modo de función, 1 SIO).

Pico W: el LED verde no está en el GPIO 25 sino en el chip Wi-Fi **CYW43439**; el código lee el Chip ID y usa `Pin('LED')`.

3. Ensamblador inline ARM Thumb

En Thumb **no se puede cargar una constante de 32 bits** con un solo `mov` inmediato. Se construye por pasos:

1. `mov(R0, 0x40)` — carga de 8 bits.
2. `lsl(R0, R0, 24)` — desplazamiento lógico de 24 bits → **0x40000000**.
3. `add` de los offsets para las direcciones de control y del SIO.

> **PAD Control (novedad de la Pico 2)**
> La Pico 2 agrega un bloque de **Control de PAD** que protege el pin de sobrecorriente; en la rutina se limita la corriente del puerto a **12 mA** (regulable de 1 a 12 mA) antes de habilitar la salida SIO.

4. Campos de bits con uctypes

Describir un registro campo por campo y manipularlo como propiedades, sin máscaras manuales (`&`, `|`, `<<`). Ejemplo: registro de control del ADC.

```
from uctypes import BF_POS, BF_LEN, UINT32, struct
ADC_CTRL_STRUCT = {
    "EN":     0 | BF_POS(0)  | BF_LEN(1) | UINT32,   # habilitar ADC
    "TS_EN":  0 | BF_POS(1)  | BF_LEN(1) | UINT32,   # sensor de temperatura
    "RROBIN": 0 | BF_POS(16) | BF_LEN(4) | UINT32,   # round-robin
}
adc_cs = struct(0x4000c000 + 0x00, ADC_CTRL_STRUCT)
adc_cs.EN = 1        # modifica solo el bit 0
```

Exige estudiar el **datasheet** (posición y longitud exactas). El profesor construyó así el acceso al **DMA** antes del soporte oficial y logró la mitad de ciclos que las soluciones convencionales.

5. Reflexión: Plan de Estudios 2026

La IA no debe enseñarse como consumo de librerías, sino con su cadena en hardware: señales y Fourier → DSP y espectrogramas → reconocimiento de patrones y redes → procesadores con unidades vectoriales y FPU.

PARTE 2 · Resumen para estudiar

Versión corta de lo que más probablemente se pregunta. Léela primero y vuelve a la Parte 1 si algo no queda claro.

> **Librerías vs. registros (examen)**
>
> `machine.Pin` reejecuta la configuración en el bucle: ≥ 4 ciclos extra por vuelta. Con `mem32` la configuración cuesta 5 ciclos fijos al inicio.

Dato	Para recordar
IO_BANK0	Pico 1: 0x40014000; Pico 2: 0x40000000
SIO	0xd0000000
GPIO25_CTRL	base + 0x060, función 5 (SIO)
OUT_ENABLE_SET / OUT_XOR	SIO + 0x024 / SIO + 0x01c, máscara 1 << 25
Thumb	Sin inmediatos de 32 bits: `mov 0x40` + `lsl 24`
PAD Control (Pico 2)	Limitar a 12 mA
uctypes	BF_POS, BF_LEN, UINT32 → campos como propiedades
Pico W	LED en el CYW43439